

TASK 3 : BREAST CANCER PREDICTION API USING FASTAPI AND DOCKER

Submitted By :

✧ **Name: Yuvraj Singh Tomar**

✧ **CID: IS-2026-9762**

✧ **Internship: Alfido Tech AI Internship**

✧ **Task Number: 3**

✧ **Project Title : BREAST CANCER PREDICTION
API USING FASTAPI AND DOCKER**

1. Introduction

Machine Learning has become an essential technology in modern healthcare by enabling computers to analyze medical data and assist in disease prediction. However, developing an accurate machine learning model is only one part of the complete process. For a model to be useful in real-world applications, it must be deployed so that users and software systems can access it easily.

This project focuses on deploying a **Breast Cancer Prediction Model** using the **FastAPI** framework. The trained Random Forest model predicts whether a breast tumor is **Benign** or **Malignant** based on thirty medical features. The application accepts patient data through an API request, preprocesses it using a saved **StandardScaler**, performs prediction using the trained model, and returns the result in JSON format.

To ensure portability and easy deployment, the project also includes **Docker containerization**, allowing the application to run consistently on different systems without dependency conflicts.

2. Problem Statement

Although machine learning models can achieve high prediction accuracy, they often remain limited to Jupyter notebooks or research environments. Such models cannot directly serve predictions to real users or applications.

The challenge addressed in this project is to convert a trained breast cancer classification model into a production-ready REST API that accepts patient information, processes it using the trained model, and returns predictions instantly. The project also aims to package the application using Docker to ensure consistent deployment across multiple environments.

3. Objectives

The main objectives of this project are:

- Deploy a trained Random Forest classification model using FastAPI.
- Load the saved machine learning model and StandardScaler.
- Accept patient information through API requests.
- Perform preprocessing before prediction.
- Return prediction results in JSON format.
- Generate automatic API documentation using Swagger UI.
- Containerize the complete application using Docker.
- Demonstrate a complete machine learning deployment workflow.

4. Technology Stack

The project utilizes several technologies to build a complete deployment pipeline.

Technology	Purpose
Python	Main programming language
FastAPI	API development
Scikit-learn	Machine learning model
Joblib	Saving and loading the model
NumPy	Numerical computations
Pydantic	Input validation
Uvicorn	FastAPI server
Docker	Containerization
Swagger UI	API testing interface

5. Dataset Description

The project uses the **Breast Cancer Wisconsin Diagnostic Dataset**, a well-known binary classification dataset available in Scikit-learn.

The dataset contains:

- **569 patient records**
- **30 numerical features**
- **2 target classes**
 - Benign
 - Malignant

The features describe various tumor characteristics such as radius, texture, perimeter, area, smoothness, concavity, compactness, and symmetry.

Before training, the dataset was preprocessed using **StandardScaler**, ensuring that all features were normalized. The trained scaler and Random Forest model were then saved using Joblib for deployment.

```
import pandas as pd
from sklearn.datasets import load_breast_cancer

cancer = load_breast_cancer()
df = pd.DataFrame(cancer.data, columns=cancer.feature_names)
df.head()
```

	mean radius	mean texture	mean perimeter	mean area	mean smoothness	mean compactness	mean concavity	mean concave points	mean symmetry	mean fractal dimension	...	worst radius	worst texture	worst perimeter	worst area	worst smoothness	worst compactness
0	17.99	10.38	122.80	1001.0	0.11840	0.27760	0.3001	0.14710	0.2419	0.07871	...	25.38	17.33	184.60	2019.0	0.1622	0.05445
1	20.57	17.77	132.90	1326.0	0.08474	0.07864	0.0869	0.07017	0.1812	0.05667	...	24.99	23.41	158.80	1956.0	0.1238	0.04003
2	19.69	21.25	130.00	1203.0	0.10960	0.15990	0.1974	0.12790	0.2069	0.05999	...	23.57	25.53	152.50	1709.0	0.1444	0.08932
3	11.42	20.38	77.58	386.1	0.14250	0.28390	0.2414	0.10520	0.2597	0.09744	...	14.91	26.50	98.87	567.7	0.2098	0.15954
4	20.29	14.34	135.10	1297.0	0.10030	0.13280	0.1980	0.10430	0.1809	0.05883	...	22.54	16.67	152.20	1575.0	0.1374	0.04023

5 rows × 30 columns

6. Model Development

The machine learning model was developed during the classification phase of the project. After preprocessing the dataset, multiple supervised learning algorithms were evaluated. Among them, the **Random Forest Classifier** produced the best overall performance and was selected for deployment.

The development process involved:

- Data preprocessing
- Train-test split
- Feature scaling
- Model training
- Model evaluation
- Saving the trained model

The model and scaler were serialized using Joblib as:

- best_classification_model.pkl
- scaler.pkl

These files are loaded whenever the API starts, eliminating the need for retraining.

```
from sklearn.ensemble import RandomForestClassifier

rf_model = RandomForestClassifier(
    n_estimators=100,
    random_state=42
)

rf_model.fit(X_train_scaled, y_train)

rf_predictions = rf_model.predict(X_test_scaled)

rf_accuracy = accuracy_score(y_test, rf_predictions)

print("Random Forest Accuracy :", round(rf_accuracy,4))

Random Forest Accuracy : 0.9561
```

7. API Implementation

The deployment component was implemented using **FastAPI**, a modern Python framework for building high-performance REST APIs.

The application consists of two endpoints:

Home Endpoint

GET /

This endpoint confirms that the API is running successfully.

Example Response:

```
{
  "message": "Welcome to the Breast Cancer Prediction API!"
}
```

Prediction Endpoint

POST /predict

The user submits thirty numerical feature values representing a patient's tumor characteristics.

The API performs the following steps:

1. Receives JSON input.
2. Validates data using Pydantic.
3. Converts data into a NumPy array.
4. Applies StandardScaler.

5. Predicts using Random Forest.
6. Calculates prediction confidence.
7. Returns the prediction.

Example Response:

```
{
  "prediction": "Benign",
  "class": 1,
  "confidence": 100,
  "model": "Random Forest",
  "status": "Prediction Successful"
}
```

FastAPI automatically generates interactive API documentation through Swagger UI, making testing simple and user-friendly.

```
1 from fastapi import FastAPI
2 from pydantic import BaseModel
3 import joblib
4 import numpy as np
5
6 # Load trained model and scaler
7 model = joblib.load("best_classification_model.pkl")
8 scaler = joblib.load("scaler.pkl")
9
10 # Create FastAPI app
11 app = FastAPI(
12     title="Breast Cancer Prediction API",
13     description="API to predict breast cancer using a trained Random Forest model.",
14     version="1.0"
15 )
16
17 # Home Endpoint
18 @app.get("/")
19 def home():
20     return {
21         "message": "Welcome to the Breast Cancer Prediction API!"
22     }
23
24 # Input Schema
25 class PatientData(BaseModel):
26     mean_radius: float
27     mean_texture: float
28     mean_perimeter: float
29     mean_area: float
30     mean_smoothness: float
31     mean_compactness: float
32     mean_concavity: float
33     mean_concave_points: float
34     mean_symmetry: float
35     mean_fractal_dimension: float
36     radius_error: float
37     texture_error: float
38     perimeter_error: float
```

```

73     data.radius_error,
74     data.texture_error,
75     data.perimeter_error,
76     data.area_error,
77     data.smoothness_error,
78     data.compactness_error,
79     data.concavity_error,
80     data.concave_points_error,
81     data.symmetry_error,
82     data.fractal_dimension_error,
83     data.worst_radius,
84     data.worst_texture,
85     data.worst_perimeter,
86     data.worst_area,
87     data.worst_smoothness,
88     data.worst_compactness,
89     data.worst_concavity,
90     data.worst_concave_points,
91     data.worst_symmetry,
92     data.worst_fractal_dimension
93 ])
94
95 input_scaled = scaler.transform(input_data)
96
97 prediction = model.predict(input_scaled)[0]
98
99 probabilities = model.predict_proba(input_scaled)[0]
100 confidence = float(max(probabilities))
101
102 result = "Malignant" if prediction == 0 else "Benign"
103
104 return {
105     "prediction": result,
106     "class": int(prediction),
107     "confidence": round(confidence * 100, 2),
108     "model": "Random Forest",
109     "status": "Prediction Successful"
110 }

```

← → ↺ 127.0.0.1:8000/docs#/ ☆ ⓘ ⋮

Breast Cancer Prediction API 1.0 OpenAPI 3.1

[/openapi.json](#)

API to predict breast cancer using a trained Random Forest model.

default

GET	/ Home
POST	/predict Predict

Schemas

HTTPValidationError > Expand all object

PatientData > Expand all object

ValidationError > Expand all object

8. Docker Containerization

To ensure portability and simplify deployment, the application was containerized using Docker.

The Dockerfile performs the following tasks:

- Uses the official Python image.
- Creates a working directory.
- Copies all project files.
- Installs dependencies.
- Exposes port **8000**.
- Starts the FastAPI application using Uvicorn.

Docker ensures that the API behaves consistently on different computers by packaging the application together with all required libraries and dependencies.

```
1 FROM python:3.11-slim
2
3 WORKDIR /app
4
5 COPY . .
6
7 RUN pip install --no-cache-dir -r requirements.txt
8
9 EXPOSE 8000
10
11 CMD ["uvicorn", "app:app", "--host", "0.0.0.0", "--port", "8000"]
```

9. Testing and Results

The API was tested using the automatically generated Swagger UI interface available at:

`http://127.0.0.1:8000/docs`

Testing confirmed that:

- The server started successfully.
- Input validation worked correctly.
- The model generated predictions accurately.
- JSON responses were returned instantly.

Sample output:

```
{
```

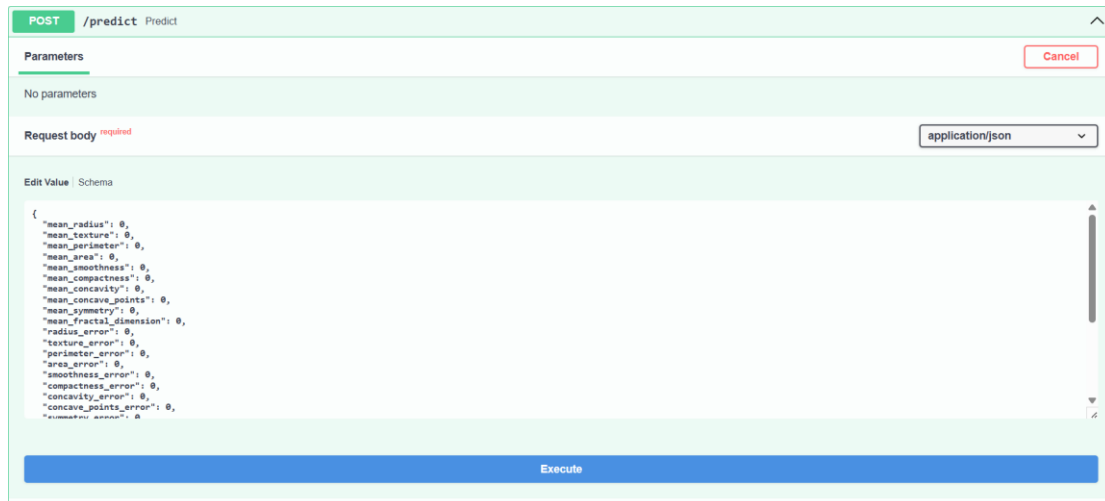


```

    "prediction": "Benign",
    "class": 1,
    "confidence": 100,
    "model": "Random Forest",
    "status": "Prediction Successful"
}

```

The confidence score provides additional information regarding the certainty of the prediction, making the API more informative than simply returning the predicted class.



POST /predict Predict

Parameters Cancel

No parameters

Request body required application/json

Edit Value | Schema

```

{
  "mean_radius": 0,
  "mean_texture": 0,
  "mean_perimeter": 0,
  "mean_area": 0,
  "mean_smoothness": 0,
  "mean_compactness": 0,
  "mean_concavity": 0,
  "mean_concave_points": 0,
  "mean_symmetry": 0,
  "mean_fractal_dimension": 0,
  "radius_error": 0,
  "texture_error": 0,
  "perimeter_error": 0,
  "area_error": 0,
  "smoothness_error": 0,
  "compactness_error": 0,
  "concavity_error": 0,
  "concave_points_error": 0,
  "symmetry_error": 0,
  "fractal_dimension_error": 0
}

```

Execute



Request URL

http://127.0.0.1:8000/predict

Server response

Code	Details
200	Response body <pre> { "prediction": "Benign", "class": 1, "confidence": 100, "model": "Random Forest", "status": "Prediction Successful" } </pre> Response headers <pre> content-length: 109 content-type: application/json date: Fri, 17 Jul 2026 05:47:47 GMT server: uvicorn </pre>

Responses

Code	Description	Links
------	-------------	-------

10. Advantages

The developed API offers several advantages:

- Real-time breast cancer prediction.
- Easy integration with external applications.
- Automatic request validation.
- Interactive Swagger documentation.
- Structured JSON responses.
- Fast prediction using a pre-trained model.
- Portable deployment using Docker.
- Reusable machine learning model.
- Easy scalability for future enhancements.

11. Limitations

Despite its successful implementation, the project has certain limitations:

- Supports only binary classification.
- Requires manual entry of all thirty features.
- Runs locally unless deployed to a cloud platform.
- Does not store prediction history.
- No user authentication or authorization.
- Limited to a single machine learning model.

12. Future Scope

The project can be enhanced in several ways:

- Deploy on cloud platforms such as AWS, Azure, or Google Cloud.
- Develop a graphical web interface.
- Build mobile applications connected to the API.
- Add user authentication.
- Store prediction history in a database.
- Implement Explainable AI techniques such as SHAP or LIME.
- Support multiple machine learning models.
- Integrate CI/CD pipelines for automated deployment.

13. Conclusion

This project successfully demonstrated the deployment of a machine learning model using **FastAPI** and **Docker**. A trained Random Forest classifier was integrated into a REST API capable of predicting breast cancer from patient data in real time. The application preprocesses incoming data using a saved `StandardScaler`, performs prediction using the trained model, and returns structured JSON responses containing the predicted class, confidence score, model information, and execution status.

The project highlights the complete machine learning deployment pipeline, including model serialization, API development, testing, and containerization. It provides valuable practical experience in transforming machine learning models into deployable software services. Furthermore, the project serves as a strong foundation for future work involving cloud deployment, healthcare applications, and scalable AI systems.

14. References

1. FastAPI Documentation – <https://fastapi.tiangolo.com/>
 2. Scikit-learn Documentation – <https://scikit-learn.org/>
 3. Docker Documentation – <https://docs.docker.com/>
 4. NumPy Documentation – <https://numpy.org/doc/>
 5. Joblib Documentation – <https://joblib.readthedocs.io/>
 6. UCI Machine Learning Repository – <https://archive.ics.uci.edu/ml/index.php>
 7. Pedregosa, F., et al. (2011). *Scikit-learn: Machine Learning in Python*. Journal of Machine Learning Research.
-